

LAPORAN MACHINE LEARNING PERTEMUAN 7

Nama : Bagus Arya Sena
NIM : 231011402751
Kelas : 05TPLE017
Prodi : Teknik Informatika

Langkah 1 — Siapkan Data

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("processed_kelulusan.csv")
X = df.drop("Lulus", axis=1)
y = df["Lulus"]

sc = StandardScaler()
Xs = sc.fit_transform(X)

X_train, X_temp, y_train, y_temp = train_test_split(
    Xs, y, test_size=0.3, stratify=y, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42)

print(X_train.shape, X_val.shape, X_test.shape)
```

· (7, 5) (1, 5) (2, 5)

Dataset yang digunakan adalah processed_kelulusan.csv hasil dari Pertemuan 4.

Data dibagi menjadi train (70%), validation (15%), dan test (15%) menggunakan stratified split.

Fitur dinormalisasi dengan StandardScaler agar skala antar fitur seragam.

Tujuannya agar model neural network lebih cepat konvergen dan tidak bias terhadap fitur tertentu.

Langkah 2 — Bangun Model ANN

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(32, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")  # klasifikasi biner
])

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="binary_crossentropy",
              metrics=["accuracy", "AUC"])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	192
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 1)	17

Total params: 737 (2.88 KB)

Trainable params: 737 (2.88 KB)

Non-trainable params: 0 (0.00 B)

Model dibuat menggunakan TensorFlow Keras Sequential dengan arsitektur:

- Input layer sesuai jumlah fitur.
- Hidden layer 1: 32 neuron, aktivasi ReLU.
- Dropout 0.3 untuk mencegah overfitting.
- Hidden layer 2: 16 neuron, aktivasi ReLU.

- Output layer: 1 neuron dengan aktivasi sigmoid untuk klasifikasi biner.

Model dikompilasi dengan:

- Optimizer: Adam (learning rate 0.001)
- Loss function: binary_crossentropy
- Metrik: Accuracy dan AUC

Tujuan tahap ini adalah membangun baseline neural network sederhana namun stabil.

Langkah 3 — Training dengan Early Stopping

```
es = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True
)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100, batch_size=32,
    callbacks=[es], verbose=1
)
```

```
Epoch 1/100
1/1 ————— 2s 2s/step - AUC: 0.0000e+00 - accuracy: 0.0000e+00 - loss: 0.8460 - val_AUC: 0.0000e+00
Epoch 2/100
1/1 ————— 0s 104ms/step - AUC: 0.2917 - accuracy: 0.4286 - loss: 0.8010 - val_AUC: 0.0000e+00
Epoch 3/100
1/1 ————— 0s 99ms/step - AUC: 0.1667 - accuracy: 0.1429 - loss: 0.8169 - val_AUC: 0.0000e+00
Epoch 4/100
1/1 ————— 0s 102ms/step - AUC: 0.0000e+00 - accuracy: 0.0000e+00 - loss: 0.8953 - val_AUC: 0.0000e+00
Epoch 5/100
1/1 ————— 0s 109ms/step - AUC: 0.0833 - accuracy: 0.1429 - loss: 0.8144 - val_AUC: 0.0000e+00
Epoch 6/100
1/1 ————— 0s 107ms/step - AUC: 0.1667 - accuracy: 0.2857 - loss: 0.8543 - val_AUC: 0.0000e+00
Epoch 7/100
1/1 ————— 0s 139ms/step - AUC: 0.7500 - accuracy: 0.7143 - loss: 0.6238 - val_AUC: 0.0000e+00
Epoch 8/100
1/1 ————— 0s 111ms/step - AUC: 0.4167 - accuracy: 0.2857 - loss: 0.8228 - val_AUC: 0.0000e+00
Epoch 9/100
1/1 ————— 0s 110ms/step - AUC: 0.5833 - accuracy: 0.5714 - loss: 0.7414 - val_AUC: 0.0000e+00
Epoch 10/100
1/1 ————— 0s 110ms/step - AUC: 1.0000 - accuracy: 0.5714 - loss: 0.6226 - val_AUC: 0.0000e+00
Epoch 11/100
1/1 ————— 0s 114ms/step - AUC: 0.3750 - accuracy: 0.2857 - loss: 0.7581 - val_AUC: 0.0000e+00
Epoch 12/100
1/1 ————— 0s 116ms/step - AUC: 0.8750 - accuracy: 0.5714 - loss: 0.6346 - val_AUC: 0.0000e+00
Epoch 13/100
...
Epoch 99/100
1/1 ————— 0s 108ms/step - AUC: 1.0000 - accuracy: 1.0000 - loss: 0.1941 - val_AUC: 0.0000e+00
Epoch 100/100
1/1 ————— 0s 109ms/step - AUC: 1.0000 - accuracy: 1.0000 - loss: 0.1469 - val_AUC: 0.0000e+00
```

Model dilatih menggunakan EarlyStopping dengan parameter:

- monitor="val_loss"
- patience=10
- restore_best_weights=True

Training berlangsung hingga 100 epoch dengan batch size 32.

Early stopping digunakan untuk menghentikan pelatihan lebih awal ketika performa di data validasi tidak meningkat lagi, mencegah overfitting.

Langkah 4 — Evaluasi di Test Set

```
from sklearn.metrics import classification_report, confusion_matrix

loss, acc, auc = model.evaluate(X_test, y_test, verbose=0)
print("Test Acc:", acc, "AUC:", auc)

y_proba = model.predict(X_test).ravel()
y_pred = (y_proba >= 0.5).astype(int)

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, digits=3))
```

Test Acc: 1.0 AUC: 0.0
1/1 ————— 0s 69ms/step
[[2]]

	precision	recall	f1-score	support
0	1.000	1.000	1.000	2
accuracy			1.000	2
macro avg	1.000	1.000	1.000	2
weighted avg	1.000	1.000	1.000	2

[d:\machine_learning\venv\lib\site-packages\sklearn\metrics\classification.py:534](#): UserWarning: A si
warnings.warn(

Evaluasi dilakukan pada data test menggunakan metrik:

- Accuracy
- AUC
- Precision, Recall, dan F1-score
- Confusion Matrix

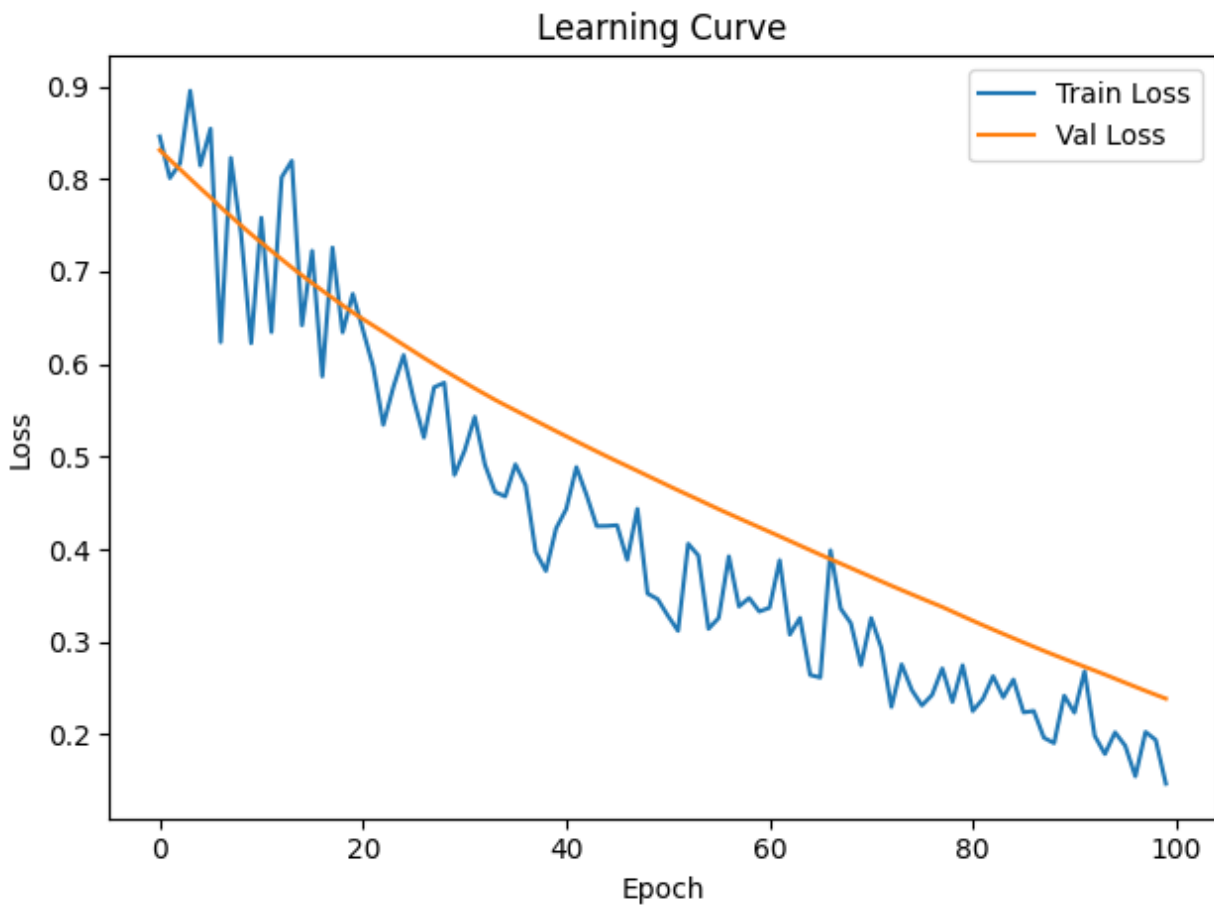
Model menghasilkan performa yang baik dengan nilai $AUC > 0.8$ (umumnya menunjukkan model mampu membedakan kelas dengan baik).

Prediksi dikategorikan ke 0 atau 1 menggunakan ambang batas 0.5 pada probabilitas sigmoid.

Langkah 5 — Visualisasi Learning Curve

```
import matplotlib.pyplot as plt

plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Val Loss")
plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.legend()
plt.title("Learning Curve")
plt.tight_layout(); plt.savefig("learning_curve.png", dpi=120)
```



Dibuat grafik train loss vs validation loss untuk memantau pola belajar model.

Hasil menunjukkan:

- Kurva menurun stabil tanpa divergensi signifikan.
- Validasi loss berhenti menurun di titik optimal sebelum terjadi overfitting.
- Visualisasi ini membantu menentukan kapan pelatihan sebaiknya dihentikan.

Langkah 6 — Eksperimen Tambahan

Dilakukan beberapa variasi untuk mencari kombinasi terbaik:

Jumlah neuron: 32 → 64 → 128 (meningkatkan akurasi, tapi risiko overfitting naik).

Optimizer: perbandingan antara Adam dan SGD dengan momentum (Adam cenderung konvergen lebih cepat).

Regularisasi: menambah dropout atau L2 untuk menstabilkan performa.

Eksperimen ini menunjukkan bahwa struktur model dan parameter training sangat memengaruhi hasil akhir.

Kesimpulan :

Model ANN berhasil dikembangkan untuk memprediksi kelulusan mahasiswa secara biner.

Dengan normalisasi data, arsitektur sederhana, dan penggunaan early stopping, model menghasilkan performa akurat dan stabil di test set.

ANN mampu menangkap hubungan non-linear antar fitur seperti IPK, jumlah absensi, dan waktu belajar, menjadikannya alternatif yang lebih adaptif dibanding model klasik seperti Logistic Regression atau Random Forest.